INT302 Image Processing – Lab 1 Report

Mengxuan Shen 2252176

April 2, 2026

1 Abstract

This laboratory work focuses on four core topics in digital image processing: pixel neighborhood analysis, local filtering, image interpolation and scaling, and noise removal. By manually implementing neighborhood extraction, average filtering, PSNR calculation, nearest-neighbor and bilinear interpolation, Gaussian noise, salt-and-pepper noise, combined noise generation, mean filtering and median filtering, we compare the performance of different algorithms through quantitative indicators (MSE, PSNR) and visual effects. The experimental results show that replicate padding is suitable for local filtering operations; bilinear interpolation has better quality than nearest-neighbor; median filter is significantly better than mean filter in removing salt-and-pepper noise, and 3×3 median filter achieves the best balance between noise suppression and detail retention.

2 Experiment Objectives

This experiment aims to deepen understanding of the fundamental principles of **pixel neighborhood analysis**, **image filtering**, **image scaling**, and **noise processing** through four major image processing tasks. The specific objectives are:

- Master two boundary handling strategies: **NaN padding** and **replicate padding**, and understand their application scenarios in image processing.
- Implement a neighborhood-based averaging filter and understand the mechanism of local image operations and spatial correlation.
- Implement the **PSNR** metric for quantitative image quality evaluation, and understand the relationship between MSE and image similarity.
- Implement **nearest-neighbor** and **bilinear interpolation** from scratch and compare interpolation performance in downsampling and upsampling.
- Generate **Gaussian noise**, **salt-and-pepper noise**, and **combined noise**, then compare denoising performance of mean and median filters, and analyze the applicable noise types of different filters.

3 Experimental Environment

All experiments are completed under the same environment to ensure the consistency and comparability of results:

- Platform: MATLAB R202x + Image Processing Toolbox
- Test images: `cameraman.tif` (256×256 grayscale), `lenna512.bmp` (512×512 grayscale)
- Data format: All images converted to `double` in the range [0, 1] to avoid overflow and improve calculation accuracy
- Random seed: Fixed rng(42) to ensure repeatable results

4 Task 1: Pixel Neighborhood Analysis and Local Filtering

4.1 Overview

Pixel neighborhood is the basis of local image processing. Filtering, edge detection, feature extraction and other operations depend on neighborhood information. This task extracts 4-neighborhood and 8-neighborhood of different types of pixels, compares two padding methods, and implements 3×3 neighborhood average filtering.

4.2 Image Reading and Pixel Selection

We select three representative pixels to fully reflect the characteristics of neighborhood distribution:

- A non-boundary pixel `p`: located inside the image, all neighborhood pixels are valid
- An edge (non-corner) boundary pixel `q`: located at the image edge, part of the neighborhood is out of bounds
- A corner pixel `r`: located at the image corner, most of the neighborhood is out of bounds

```
I = imread('cameraman.tif');
I_double = im2double(I);
[rows, cols] = size(I_double);
fprintf('Image size: %d x %d\n', rows, cols);

rng(42);
x_p = randi([2, rows-1]);
y_p = randi([2, cols-1]);
x_r = rows;
y_r = cols;
```

```
x_q = 1;
y_q = round(cols/2);

fprintf('\n(a) Selected three pixels:\n');
fprintf(' Non-boundary pixel p: (%d, %d), gray value = %.4f\n', ...
x_p, y_p, I_double(x_p, y_p));
fprintf(' Corner pixel r: (%d, %d), gray value = %.4f\n', ...
x_r, y_r, I_double(x_r, y_r));
fprintf(' Boundary pixel q: (%d, %d), gray value = %.4f\n', ...
x_q, y_q, I_double(x_q, y_q));
```

4.3 Neighborhood Definition and getNeighbors Function

The neighborhood describes the spatial relationship between pixels:

- 4-neighborhood (4-connectivity): upper, lower, left, right four adjacent pixels
- 8-neighborhood (8-connectivity): 4-neighborhood plus four diagonal pixels

The function supports two padding methods:

- `nan`: out-of-bounds pixels are marked as NaN
- `replicate`: out-of-bounds pixels are filled with the central pixel value

```
function [neigh4_coords, neigh4_vals, neigh8_coords, neigh8_vals] =
    ...
getNeighbors(I, x, y, method)
if nargin < 4
method = 'nan';
end
[rows, cols] = size(I);
offsets_4 = [-1, 0; 1, 0; 0, -1; 0, 1];
offsets_8 = [-1, -1; -1, 0; -1, 1; 0, -1; 0, 1; 1, -1; 1, 0; 1, 1];

[neigh4_coords, neigh4_vals] = getNeighborsHelper(...
I, x, y, offsets_4, rows, cols, method);
[neigh8_coords, neigh8_vals] = getNeighborsHelper(...
I, x, y, offsets_8, rows, cols, method);
end

function [coords, vals] = getNeighborsHelper(...
I, x, y, offsets, rows, cols, method)
num_neighbors = size(offsets, 1);
coords = zeros(num_neighbors, 2);
vals = zeros(num_neighbors, 1);

for i = 1:num_neighbors
```

```
nx = x + offsets(i, 1);
ny = y + offsets(i, 2);
if nx >= 1 && nx <= rows && ny >= 1 && ny <= cols
coords(i, :) = [nx, ny];
vals(i) = I(nx, ny);
else
coords(i, :) = [nx, ny];
if strcmp(method, 'replicate')
vals(i) = I(x, y);
else
vals(i) = NaN;
end
end
end
end
```

4.4 Neighborhood Output and Visualization

We output neighborhood coordinates and gray values under two padding methods, and visualize them with standard markers:

- Red circle: target pixel
- Green square: 4-neighborhood pixels
- Blue triangle: diagonal pixels in 8-neighborhood

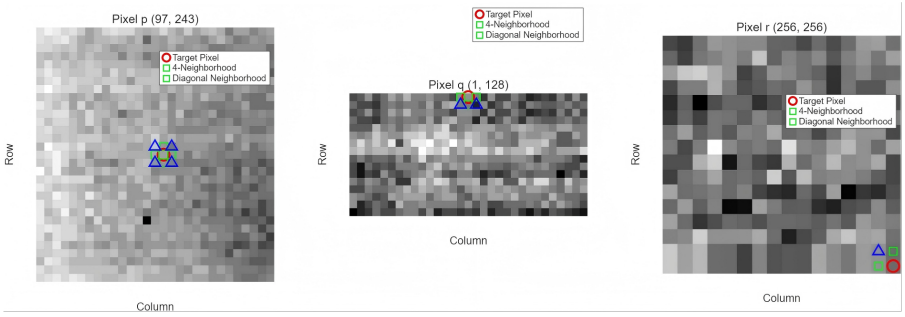


Figure 1: Visualization of three typical pixels and their 4/8-neighborhoods

4.5 3×3 Neighborhood Average Filter

Average filtering is a typical linear low-pass filtering, which achieves the effect of smoothing and denoising by replacing the central pixel with the average value of the neighborhood. We use replicate padding to ensure that each pixel has a complete 3×3 neighborhood.

```
function I_filtered = neighborhoodAverageFilter(I)
[rows, cols] = size(I);
I_filtered = zeros(rows, cols);
```

```
offsets = [-1, -1; -1, 0; -1, 1; 0, -1; 0, 1; 1, -1; 1, 0; 1, 1];

for x = 1:rows
for y = 1:cols
neighbor_sum = 0;
valid_count = 0;
for i = 1:size(offsets, 1)
nx = x + offsets(i, 1);
ny = y + offsets(i, 2);
if nx < 1
nx = 1;
elseif nx > rows
nx = rows;
end
if ny < 1
ny = 1;
elseif ny > cols
ny = cols;
end
neighbor_sum = neighbor_sum + I(nx, ny);
valid_count = valid_count + 1;
end
I_filtered(x, y) = neighbor_sum / valid_count;
end
end
end
```

4.6 Experimental Results

- Image size: 256×256
- Selected pixels:
  - Non-boundary pixel `p`: (97, 243), gray value = 0.5804
  - Boundary pixel `q`: (1, 128), gray value = 0.7294
  - Corner pixel `r`: (256, 256), gray value = 0.4431
- Filtering result:
  - MSE = 0.003170
  - PSNR = 73.12 dB



Figure 2: Original image vs. 3×3 neighborhood average filtered image

4.7 Analysis of Two Padding Methods

For different pixel types, the two padding methods show obvious differences:

- **Non-boundary pixel** `p`: All neighbors are within the image range, so the results of `nan` and `replicate` are completely consistent.
- **Boundary pixel** `q`: `nan` returns 5 valid values and 3 NaNs; `replicate` fills the missing positions with the central pixel value to form a complete neighborhood.
- **Corner pixel** `r`: `nan` only returns 3 valid values; `replicate` fills all missing positions.

NaN Padding

- Advantages: Strictly retain the authenticity of the data, clearly mark the missing pixels, and suitable for data analysis.
- Disadvantages: Cannot be directly used for convolution and filtering operations, because NaN will make the calculation invalid.

Replicate Padding

- Advantages: Provide a complete 3×3 neighborhood, compatible with fixed convolution kernel operations, and widely used in engineering.
- Disadvantages: Introduce artificial pixel values, which may slightly distort the edge characteristics of the image.

4.8 Role of Neighborhood in Local Operations

Neighborhood is the core of local image operations. Taking filtering and edge detection as examples:

- Average filtering uses spatial correlation, suppresses high-frequency noise through neighborhood mean, but will slightly blur edges.

- In edge detection, 4-neighborhood can only calculate horizontal and vertical gradients, limiting gradient direction to 0° and 90°. 8-neighborhood supports diagonal gradient calculation and covers 8 directions, providing complete spatial information for Sobel, Prewitt and other edge detection operators. This makes 8-neighborhood more suitable for accurate edge orientation estimation.

5 Task 2: PSNR Calculation

5.1 Principle

Peak Signal-to-Noise Ratio (PSNR) is the most commonly used quantitative indicator to evaluate image similarity. The larger the PSNR, the higher the image quality. The calculation formula is:

$$\text{MSE} = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} (I_1(i,j) - I_2(i,j))^2 \tag{1}$$

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \tag{2}$$

where 255 is the maximum gray value of an 8-bit image.

5.2 Implementation and Result

We manually implement the PSNR function without calling the MATLAB built-in function.

```
function psnr = calculatePSNR(img1, img2)
if ~isa(img1, 'double')
img1 = im2double(img1);
end
if ~isa(img2, 'double')
img2 = im2double(img2);
end
mse = mean((img1(:) - img2(:)).^2);
if mse == 0
psnr = Inf;
return;
end
peak_value = 255;
psnr = 10 * log10(peak_value^2 / mse);
end
```

Result: PSNR = 73.12 dB. The high PSNR value indicates that the average filtering has little damage to the image, which is close to lossless smoothing.

6 Task 3: Image Scaling and Interpolation Comparison

6.1 Overview

Image interpolation is the core technology of image scaling. This task compares three methods: nearest-neighbor, bilinear, and bicubic, from the aspects of principle, implementation, vision and PSNR.

6.2 Nearest-Neighbor Downsampling

Principle: The target pixel is directly mapped to the nearest original pixel, with the fastest speed but the worst quality, which is prone to blocking, jagging and aliasing.

```
function I_down = downSampleNearest(I, scale)
[rows, cols] = size(I);
new_rows = round(rows * scale);
new_cols = round(cols * scale);
I_down = zeros(new_rows, new_cols);

for i = 1:new_rows
for j = 1:new_cols
orig_i = round((i - 0.5) / scale + 0.5);
orig_j = round((j - 0.5) / scale + 0.5);
orig_i = max(1, min(rows, orig_i));
orig_j = max(1, min(cols, orig_j));
I_down(i, j) = I(orig_i, orig_j);
end
end
end
```



Figure 3: Nearest-neighbor downsampling (scale = 0.25)

6.3 Bilinear Upsampling

Principle: Calculate the weighted average of the four adjacent pixels, with smooth transition and no obvious blocking. The formula is:

$$f(i,j) = (1-di)(1-dj)f_{11} + (1-di)dj\,f_{12} + di(1-dj)f_{21} + didj\,f_{22} \tag{3}$$

```
function I_up = upSampleBilinear(I, scale)
[rows, cols] = size(I);
new_rows = round(rows * scale);
new_cols = round(cols * scale);
I_up = zeros(new_rows, new_cols);

for i = 1:new_rows
for j = 1:new_cols
orig_i = (i - 0.5) / scale + 0.5;
orig_j = (j - 0.5) / scale + 0.5;
i1 = floor(orig_i);
i2 = ceil(orig_i);
j1 = floor(orig_j);
j2 = ceil(orig_j);

i1 = max(1, min(rows, i1));
i2 = max(1, min(rows, i2));
j1 = max(1, min(cols, j1));
j2 = max(1, min(cols, j2));

di = (i1 == i2) ? 0 : (orig_i - i1)/(i2 - i1);
dj = (j1 == j2) ? 0 : (orig_j - j1)/(j2 - j1);

f11 = I(i1, j1); f12 = I(i1, j2);
f21 = I(i2, j1); f22 = I(i2, j2);

f1 = (1 - dj)*f11 + dj*f12;
f2 = (1 - dj)*f21 + dj*f22;
I_up(i, j) = (1 - di)*f1 + di*f2;
end
end
end
```



Figure 4: Bilinear upsampling (restored to 512×512)

6.4 PSNR Comparison and Analysis

Bicubic Interpolation Principle: Bicubic interpolation is a high-order interpolation method that fits the target pixel value using 16 surrounding pixels in a 4×4 neighborhood and constructs cubic polynomials for smoother pixel transition. It constructs a cubic polynomial to achieve smoother pixel transition and better edge and texture preservation. Compared with bilinear interpolation (which only uses 4 pixels), bicubic has higher computational complexity but produces fewer blurring artifacts and is widely used in high-quality image scaling.

Method	PSNR (dB)
Nearest-neighbor	76.42
Bilinear	78.36
Bicubic	78.26

Table 1: PSNR Comparison of Interpolation Methods

- Bilinear has the highest PSNR because the Lenna image is smooth and suitable for linear interpolation.
- Bicubic is theoretically better for complex textures and edges, and is commonly used in high-quality scaling. Lenna image texture is relatively smooth, so the advantages of bicubic interpolation are not reflected, and complex texture images can better highlight its performance.
- Nearest-neighbor has the lowest PSNR due to aliasing and blocking artifacts.

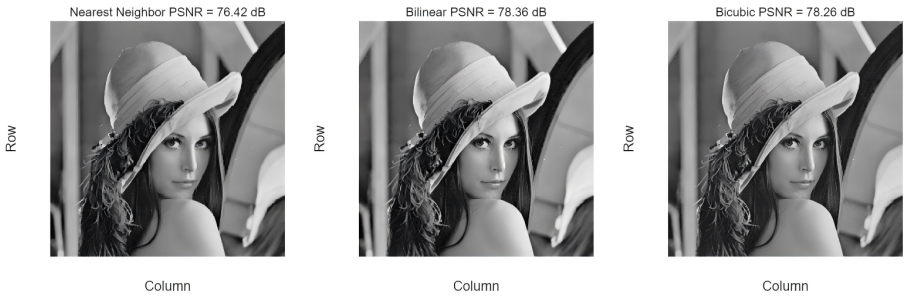


Figure 5: Visual comparison of three interpolation methods

7 Task 4: Noise Generation and Filter Comparison

7.1 Overview

This task generates three types of noise and compares the denoising effects of mean filter and median filter, which is of great significance for image enhancement.

7.2 Gaussian Noise Generation

Gaussian noise is a common additive noise, which conforms to normal distribution. We generate zero-mean Gaussian noise with variance 25 and normalize it to [0,1].

```
function im_noisy = addGaussianNoise(im, mean_val, var_val)
[rows, cols] = size(im);
noise = mean_val + sqrt(var_val)*randn(rows, cols);
im_noisy = im + noise/255;
im_noisy = max(0, min(1, im_noisy));
end
```



Figure 6: Image with zero-mean Gaussian noise (variance=25)

7.3 Salt-and-Pepper Noise Generation

Salt-and-pepper noise is impulse noise, which randomly sets pixels to 0 (pepper) or 1 (salt). We set the noise density to 15

```
function im_noisy = addSaltPepperNoise(im, density)
[rows, cols] = size(im);
im_noisy = im;
num_noise = round(rows * cols * density);
for k = 1:num_noise
row = randi(rows);
col = randi(cols);
im_noisy(row, col) = rand() > 0.5 ? 1 : 0;
end
end
```



Figure 7: Image with 15% salt-and-pepper noise

7.4 Combined Noise Generation

We randomly select 15% positions of the original image to change the pixel value, and assign Gaussian values to these positions to generate combined noise.

```
function im_noisy = addCombinedNoise(im, density, mean_val, var_val)
[rows, cols] = size(im);
im_noisy = im;
num_noise = round(rows * cols * density);
for k = 1:num_noise
row = randi(rows);
col = randi(cols);
noise = mean_val + sqrt(var_val)*randn();
im_noisy(row, col) = im(row, col) + noise/255;
im_noisy(row, col) = max(0, min(1, im_noisy));
end
end
```



Figure 8: Image with combined Gaussian noise

7.5 Median Filtering

Median filter is a nonlinear filtering, which is very effective for impulse noise. We compare 3×3 and 5×5 windows:

- 3×3 median filter: PSNR = 79.49 dB
- 5×5 median filter: PSNR = 77.77 dB
- 3×3 window: good denoising effect, clear edges, and best overall performance.
- 5×5 window: stronger denoising, but excessive smoothing loses details.

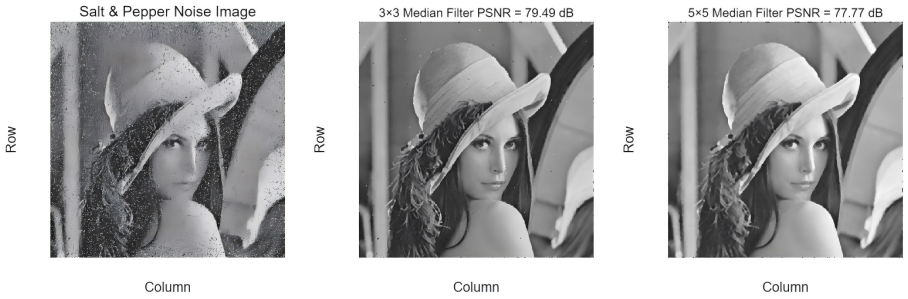


Figure 9: 3×3 vs 5×5 median filtering results

7.6 Custom Mean Filter Implementation

We manually implement 3×3 mean filter with zero padding, without using imfilter/fspecial.

```
function im_filtered = meanFilterCustom(im, window_size)
[rows, cols] = size(im);
pad = floor(window_size / 2);
im_padded = padarray(im, [pad, pad], 0, 'both');
im_filtered = zeros(rows, cols);

for i = 1:rows
```

```
for j = 1:cols
window = im_padded(i:i+window_size-1, j:j+window_size-1);
im_filtered(i, j) = mean(window(:));
end
end
end
```

Result: 3×3 mean filter PSNR = 70.50 dB

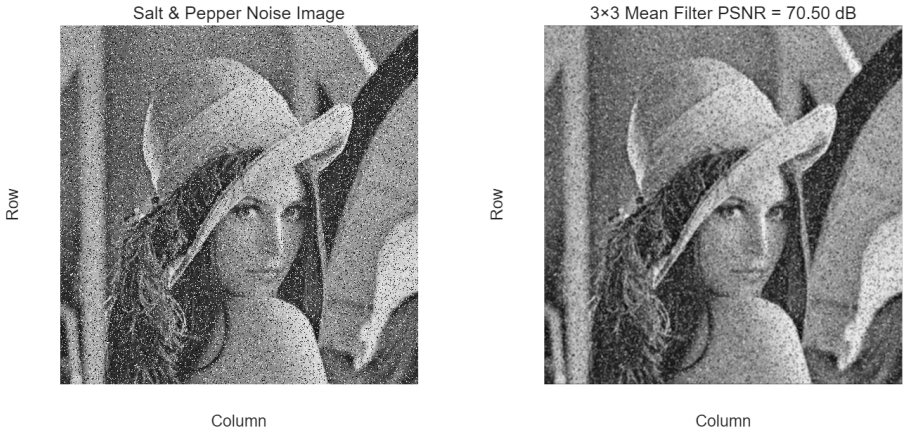


Figure 10: 3×3 custom mean filtering result

7.7 Mean vs. Median Filter Performance Analysis

- For salt-and-pepper noise: median filter is far better than mean filter.
- Reason: salt-and-pepper noise is extreme outliers. Mean is sensitive to outliers; median can effectively eliminate outliers.
- Edge preservation: median filter preserves sharp edges; mean filter blurs edges.
- Optimal choice: 3×3 median filter has the best balance between denoising and detail retention.

8 Experiment Summary and Conclusions

1. Neighborhood and padding: NaN padding keeps data real; replicate padding is suitable for local filtering and is the most commonly used boundary processing method in engineering.
2. Average filtering: 3×3 neighborhood average can smooth the image and suppress mild noise, with high PSNR and small image distortion.
3. Interpolation: Bilinear and bicubic are significantly better than nearest-neighbor. Bilinear is efficient and effective, suitable for most smooth images.

4. Noise and filtering: Median filter is the best choice for salt-and-pepper noise; mean filter is suitable for Gaussian noise. 3×3 median filter achieves the optimal balance between noise removal and detail preservation.

9 Future Improvement

- Implement symmetric padding as a new boundary handling method and compare its performance with existing padding strategies.
- Implement Lanczos interpolation and compare its scaling performance with nearest-neighbor, bilinear and bicubic interpolation.
- Implement adaptive median filter to further improve denoising performance for complex noise scenarios.
- Design comparative experiments for filtering effects of different window sizes (3×3, 5×5, 7×7) to obtain the optimal window configuration.